CNN versus ANN

Contents

1. Overview 2
2. Prerequisites 2
3. The same task, two networks 2
4. How they are similar: a filter is a node 3
5. How they differ: the parameter count 5
   5.1 Counting the parameters of a convolution layer 5
   5.2 The image size does not matter 6
6. The comparison on real data 8
7. Summary 10
8. Sources 10
9. Key terms 10

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Convolutional neural network (CNN).
- **Compare with:** Multi-layer perceptron (MLP).

1. Overview

Key point: A filter in a CNN (convolutional neural network) does what a node in an ANN (artificial neural network, with a weight from every input to every node) does, a weighted sum plus a bias, then an activation, but on a small window at a time, sliding across the image with the same weights. So a convolution layer's parameter count depends only on its filters, never on the image size, and a small CNN beats a larger ANN on images.

An ANN on images has three problems (see why not just use an ANN on images):

1. too many weights;
2. overfitting (fitting the training data so closely that new data is predicted worse);
3. the lost arrangement of the pixels.

Having built a CNN from its parts (convolution, padding and strides, **pooling** (G-1521), **LeNet-5** (G-1080)), we can now compare the two kinds of network directly: how they are alike, and how they differ.

The similarity matters for the next Notes: if a **filter** (G-777; a small grid of weights, slid over the image) works like a node, then the backpropagation learned for ANNs carries over to CNNs (see the four derivatives of a CNN).

2. Prerequisites

- Flatten and counting parameters: an ANN on images.
- Filters and feature maps, and colour images.
- Counting the parameters of LeNet-5: the CNN architecture and its parameter count.

3. The same task, two networks

Key point: The ANN flattens the 28×28 image to 784 inputs and connects all of them to every node. The CNN keeps the image 2D, convolves it with filters, adds a bias, applies ReLU, pools, and only then flattens.

Take **MNIST** (G-1249; a collection of 28×28 greyscale images of handwritten digits), and the task of classifying them into the 10 digits.

- **ANN:** the 28×28 grid is flattened into one list of 784 inputs. Hidden layers follow, as many as we like, all fully connected (every node takes every input), and a **softmax output layer** (G-1832) (a layer that turns 10 scores into 10 class probabilities) gives the class.
- **CNN:** the image stays 2D. A 3×3 filter convolves it into a 26×26 **feature map** (G-766; the grid of values the filter produces at all positions, whose size rule is in the size of the feature map). Each filter has one bias, added to every value of its feature map; with a bias of 1, every value goes up by 1. The result goes through an activation such as ReLU (keeps positive values, turns negative ones to 0), then **max pooling** (G-1182) (keeps the largest value of each small window, shrinking the map), then Flatten, fully connected layers and the softmax output.

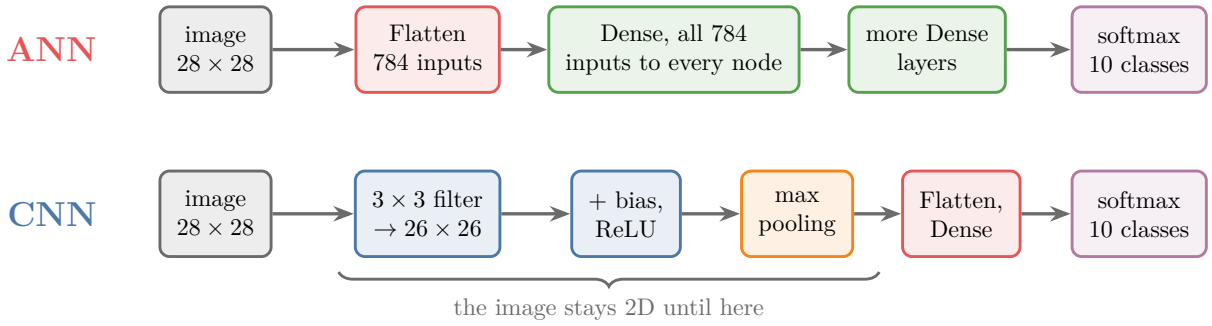


Figure 1: The two networks on the same MNIST image. The ANN flattens at once; the CNN keeps the 2D grid through convolution, ReLU and pooling, and flattens only before its last layers.

Figure 1 draws each network as a row of boxes, one per step, from the image on the left to the class on the right. Compare where the red Flatten box sits: first in the ANN, near the end in the CNN. Everything before it in the CNN works on the 2D image.

4. How they are similar: a filter is a node

> **Key point:** An ANN node computes $\mathrm{ReLU}(w_1x_1 + \cdots + w_{784}x_{784} + b)$ over all inputs at once. A filter computes $\mathrm{ReLU}(w_1x_1 + \cdots + w_9x_9 + b)$ over the 9 pixels under it, then slides and repeats with the same weights.

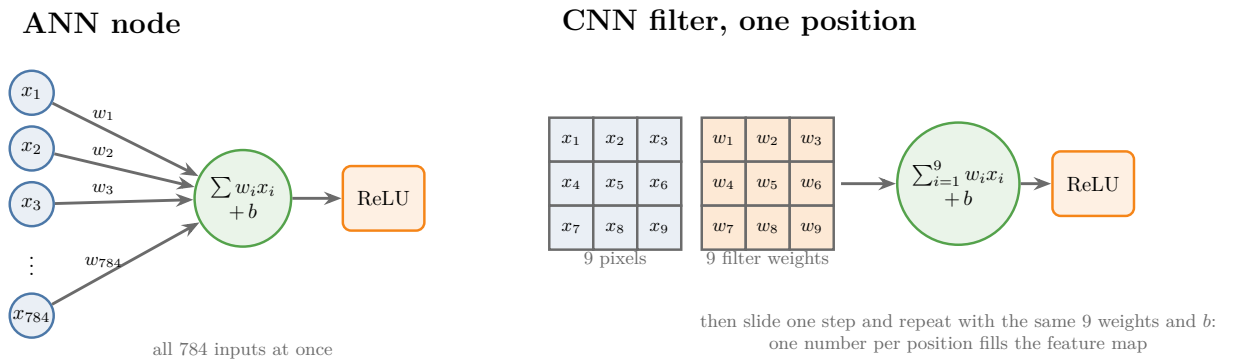


Figure 2: Left: an ANN node takes all 784 inputs, forms a weighted sum, adds its bias and applies ReLU. Right: a 3×3 filter does the same with the 9 pixels under it, then slides to the next position and repeats with the same 9 weights and bias

Look at what one node of the ANN does (Figure 2, left). Its inputs are $x_1, x_2, \ldots, x_{784}$. The node multiplies each by a weight and adds them up, adds its bias, and passes the result through an activation function:

$$a = \text{ReLU}(w_1 x_1 + w_2 x_2 + \cdots + w_{784} x_{784} + b)$$

Now look at a filter at one position (Figure 2, right). The filter covers 9 pixels, x_1 to x_9. The filter's values are trainable weights, so it computes $w_1 x_1 + \cdots + w_9 x_9$, adds the filter's bias, and passes the result through ReLU:

$$a = \text{ReLU}(w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9 + b)$$

Both compute the same thing: a dot product (multiply matching numbers, then add), plus a bias, through an activation. The Notebook checks it: the top-left value of a Keras `Conv2D` layer (Keras's convolution layer) with ReLU equals a node with 9 inputs, given the same 9 weights and bias (0.93582 both).

The difference is in how the inputs are taken. A node uses all its inputs at once. A filter takes them in chunks, one window at a time, and slides across the image, repeating the same calculation with the same weights. Sliding is how the filter captures 2D patterns, the spatial arrangement of the pixels.

Figure 3 shows the change from one to the other on a small 8×8 digit. First a node takes all 64 pixels, with one weight each. Then the same node is given only the 9 pixels under a 3×3 window. Then the window slides: watch the 9 orange lines move with it, while the caption still says 9 weights and 1 bias. Using the same weights at every position is **parameter sharing** (G-1447).

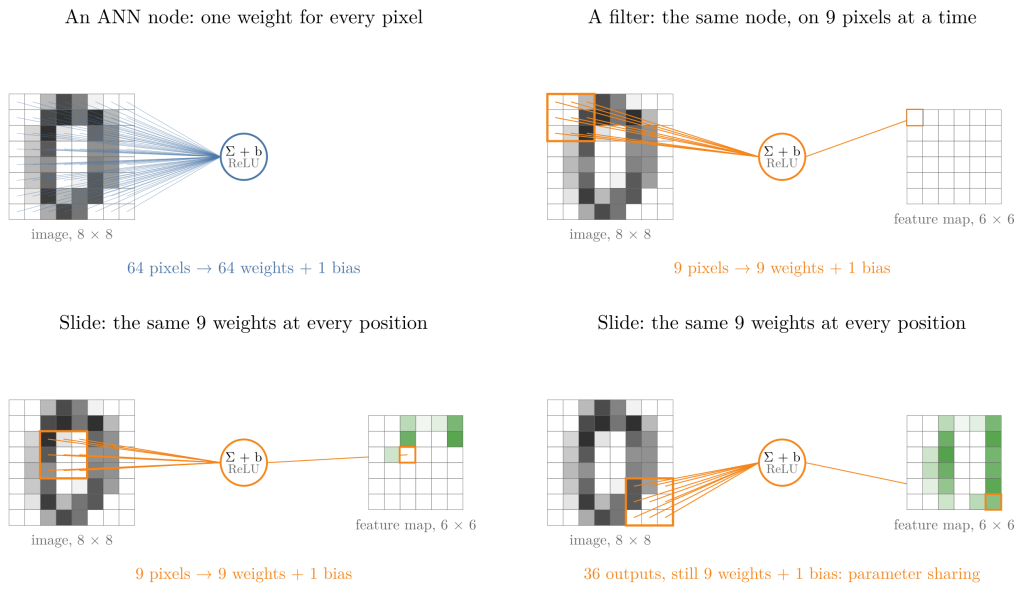


Figure 3: An ANN node with one weight per pixel (blue), then a filter: the same node on the 9 pixels under a 3 × 3 window (orange). The window slides over all 36 positions with the same 9 weights and 1 bias, and each position fills one cell of the feature map (green)

So, as a shortcut: **a filter of a CNN works like a node of an ANN**. In an ANN we train the weights of the nodes; in a CNN we train the values of the filters. Adding one more filter, with its own bias, is like adding one more node.

> **Extra:** Goodfellow et al. (2016, §9.2) name the two differences. **Sparse interactions:** (G-1842) each output depends only on a small window of the input, because the kernel (another name for the filter) is smaller than the input. **Parameter sharing:** (G-1447) the same weights are used at every position, instead of a separate weight for every input-output pair. Both reduce the number of parameters.

5. How they differ: the parameter count

> **Key point:** 50 filters of 3 × 3 × 3 (height 3, width 3, 3 colour channels) have 1,400 parameters (50 × 27 weights plus 50 biases), whether the image is 224 × 224 or 1080 × 1080. A Dense layer of 100 nodes on the same flattened images needs 15 million or 350 million.

5.1 Counting the parameters of a convolution layer

> **Key point:** Each filter: height × width × channels weights, plus 1 bias.

1. **In words:** count the weights in one filter, add its bias, multiply by the number of filters.

2. **Formula:**

$$\text{parameters} = (f \times f \times c + 1) \times k$$

for k filters of size $f \times f$ on an input with c channels.

3. **Example:** an RGB image of $224 \times 224 \times 3$ (height 224, width 224, 3 colour channels: red, green, blue) and 50 filters of $3 \times 3 \times 3$. One filter has 27 weights, and each filter has one trainable bias.

$$3 \times 3 \times 3 = 27$$

$$50 \times 27 = 1{,}350$$

$$1{,}350 + 50 = 1{,}400$$

The total is 1,400 **learnable parameters** (G-1065). The output is a $222 \times 222 \times 50$ volume.

5.2 The image size does not matter

> **Key point:** Change the image to $1080 \times 1080 \times 3$ and the answer is still 1,400.

Now take a much bigger image, $1080 \times 1080 \times 3$, with the same 50 filters of $3 \times 3 \times 3$. The number of **learnable parameters** (G-1065) is still 1,400: the weights and biases belong to the filters, and the filters do not depend on the size of the input. A bigger image only means more positions for the same filters to slide over.

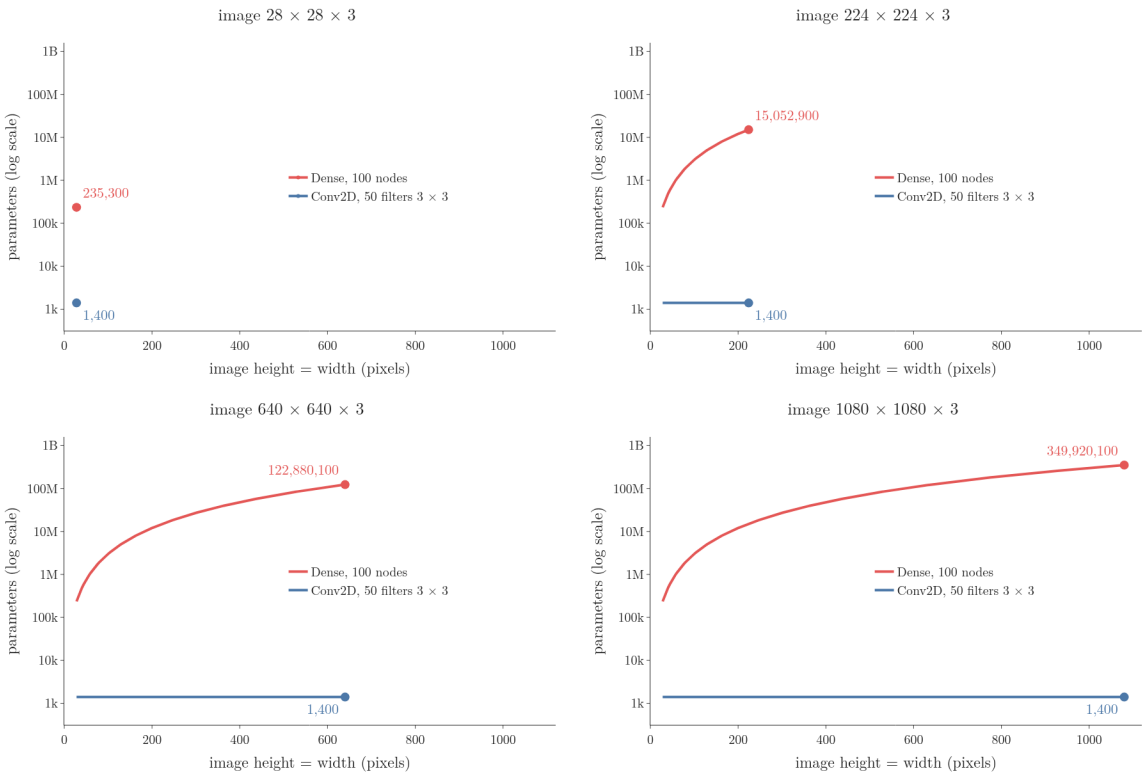


Figure 4: The image grows from 28 × 28 × 3 to 1080 × 1080 × 3. Blue: parameters of a Conv2D layer of 50 filters of 3 × 3. Red: parameters of a Dense layer of 100 nodes on the flattened image. Log scale.

Figure 4 is an animation: the image grows, and the vertical axis (a log scale: each gridline is ten times the one below) shows the parameter count. Watch the two lines as the image grows: the blue line stays flat at 1,400, while the red line climbs with the number of pixels. For an $n \times n \times 3$ image, each of the 100 nodes has $n \times n \times 3$ weights and 1 bias, so the count is:

$$(n \times n \times 3 + 1) \times 100$$

At 1080 × 1080 it passes 349 million (table below).

An ANN behaves very differently. Flattening gives 224 × 224 × 3 inputs, and a hidden layer of y nodes needs (inputs × y) weights. If the inputs grow to 1080 × 1080 × 3, the weights grow with them, into the millions. The Notebook builds both layers in Keras:

Image	Conv2D, 50 filters of 3 × 3	Dense, 100 nodes on the flattened image
224 × 224 × 3	1,400	15,052,900
1080 × 1080 × 3	1,400	349,920,100

This independence from the image size answers two of the three problems of an ANN on images. Weights that do not grow with the image keep the network small (problem 1), and fewer weights leave less room to **overfit** (G-1429; fitting the training data too closely) (problem 2). The third problem, the lost spatial arrangement, is answered by the sliding window itself: a filter always sees neighbouring pixels together, so it captures 2D patterns.

The fully connected part of a CNN does depend on the image size, since Flatten's output grows with the image. The growing Flatten output is why most of **LeNet-5** (G-1080)'s 61,706 parameters (48,120) sit in its first Dense layer (see counting the parameters of LeNet-5), and why **pooling** (G-1521), which shrinks the maps before Flatten, matters.

6. The comparison on real data

> **Key point:** With 8 times fewer parameters, a small CNN beats the ANN on MNIST (98.82% against 97.80%) and on Fashion-MNIST (89.08% against 88.22%), and it overfits less.

The Notebook trains two networks on the same data, with the same optimizer (Adam, a variant of gradient descent), 15 epochs (full passes over the training data) and batch size 128 (observations per weight update), 3 seeds each:

- **ANN:** Flatten, Dense(128, ReLU), Dense(10, softmax) (Dense is a fully connected layer: the number is its nodes), as in the MNIST network: 101,770 parameters.
- **CNN:** Conv2D(16 filters, 3×3, ReLU), MaxPooling 2×2, Conv2D(32 filters, 3×3, ReLU), MaxPooling 2×2, Flatten, Dense(10, softmax): 12,810 parameters.

The data are MNIST and **Fashion-MNIST** (G-756), a dataset of the same format (28×28 greyscale images, 10 classes, 60,000 training and 10,000 test images) showing clothing items such as shirts, sneakers and bags (Keras documentation). It is harder: both networks score about 10 points lower on it than on MNIST.

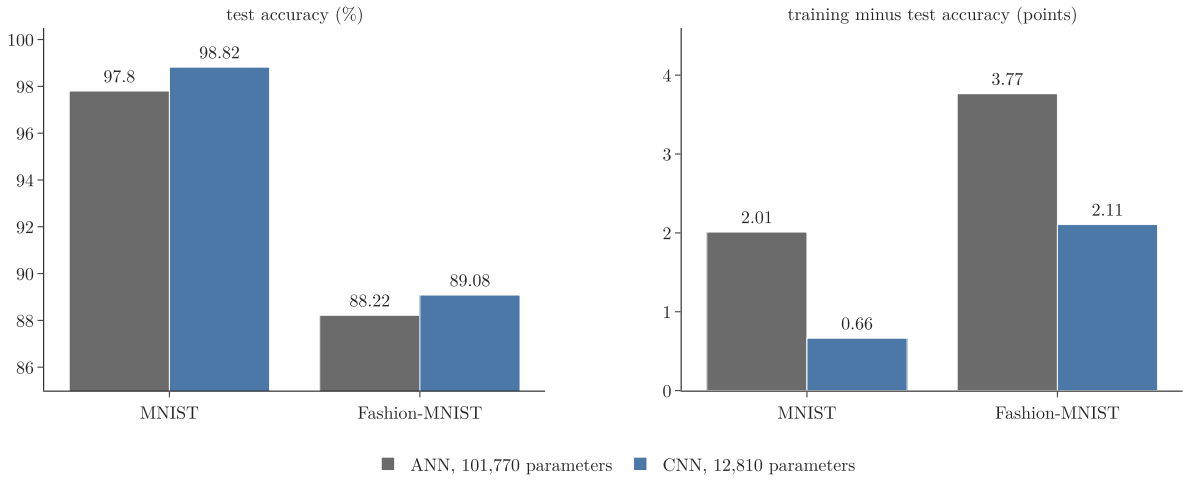


Figure 5: Mean of 3 seeds. Left: test accuracy (the axis starts at 85%). Right: training accuracy minus test accuracy, a measure of overfitting. The CNN, with 8 times fewer parameters, is more accurate and overfits less on both datasets

Figure 5 shows two bar charts, with a pair of bars (ANN, CNN) for each dataset. In the left chart a taller bar is better; in the right chart a shorter bar is better, because a small gap means little overfitting. The Notebook gives:

	MNIST	Fashion-MNIST
ANN test accuracy (101,770 parameters)	97.80%	88.22%
CNN test accuracy (12,810 parameters)	98.82%	89.08%
ANN training minus test accuracy	2.01 points	3.77 points
CNN training minus test accuracy	0.66 points	2.11 points

On both datasets every CNN run beats every ANN run (Notebook). The CNN also has a smaller gap between training and test accuracy: the ANN fits its training data more closely than it generalises, while the CNN's shared filters generalise better.

> **Extra:** Fewer parameters does not mean fewer calculations. Each of the CNN's filter weights is reused at every position, so per image the CNN above performs 662,912 multiplications against the ANN's 101,632 (Notebook). Compared with a fully connected layer producing an output of the same size, though, a convolution needs far fewer operations, because each output uses only a small window (Goodfellow et al. 2016, §9.2).

7. Summary

	ANN	CNN
Input	flattened to 1D	kept 2D (or 3D with channels)
Basic unit	node: weighted sum of all inputs + bias, activation	filter: weighted sum of a window + bias, activation, slid over the image
What is trained	weights and biases of the nodes	filter values and one bias per filter
Parameters of the first layer	inputs × nodes: grows with the image	$(f \times f \times c + 1) \times k$: independent of the image
Spatial arrangement	lost	captured by the sliding window
On MNIST / Fashion-MNIST (Notebook)	97.80% / 88.22% with 101,770 parameters	98.82% / 89.08% with 12,810 parameters

- A filter is to a CNN what a node is to an ANN: dot product, bias, activation, so the backpropagation learned for ANNs carries over to CNNs.
- A convolution layer's parameters depend on the number and size of its filters, not on the image size, because the same filter weights are reused at every position (**parameter sharing** (G-1447)).
- So CNNs need fewer parameters, overfit less and use the 2D structure of images, which is why a small CNN beat a larger ANN on both datasets.

8. Sources

Built from

- CampusX, "Comparing CNN Vs ANN | CampusX", YouTube, https://www.youtube.com/watch?v=niE

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §9.2 (sparse interactions, parameter sharing).
- Keras API documentation: `Conv2D`, `Dense`, and the Fashion-MNIST dataset (`keras.datasets.fashion_mnist`), keras.io/api.
- Xiao, H., Rasul, K. and Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Sparse interactions (G-1842)	Each output of a convolution depends only on a small window of the input, because the filter is small; so a convolution needs far fewer weights than a fully connected layer.
Learnable (trainable) parameters (G-1065)	The weights and biases that training changes; for a convolution layer, the filter values and one bias per filter.
Fashion-MNIST (G-756)	A dataset of 28 × 28 greyscale images of 10 kinds of clothing, in the same format as MNIST.
Pooling (G-1521)	Replacing each small window of a feature map with one summary number, to shrink the map.
LeNet-5 (G-1080)	An early network for reading handwritten digits, built by LeCun et al. in 1998 (a CNN): two blocks of convolution then pooling (conv-pool blocks), then layers of 120, 84 and 10 nodes.
Filter (kernel) (G-777)	A small grid of weights slid over the image; at each position it multiplies and adds, so it responds strongly where its pattern (such as an edge) appears and produces a feature map. Its depth equals the input's number of channels.
MNIST (G-1249)	A dataset of about 70,000 handwritten-digit images of 28 × 28 pixels.
Softmax output layer (G-1832)	An output layer with one node per class whose outputs are probabilities adding up to 1.
Feature map (CNN) (G-766)	The grid of numbers a filter produces; large where the filter's pattern is present.
Max pooling (G-1182)	Pooling that keeps only the largest value in each window (usually 2 × 2), which shrinks the feature map while keeping the strongest response in each region.
Parameter sharing (G-1447)	Using the same weights at every position or time step; it lets a model handle sequences of any length and recognise a pattern wherever it appears.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.